APPS/WEB · REFACTOR AUDIT · 28 SEP 2026

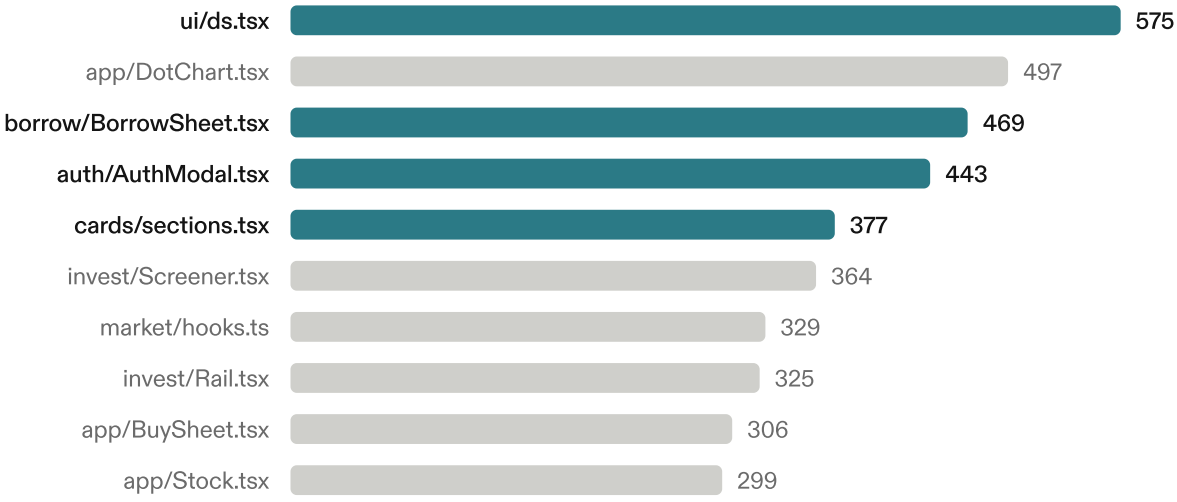
De-Slop Report

Where the web app's code carries weight it doesn't need, and a step-by-step plan to remove it. The code was read in full; nothing has been changed yet.

0	33	2	No
uses of `any`	files with more than one component	components over 400 lines mixing logic and UI	Zustand needed today

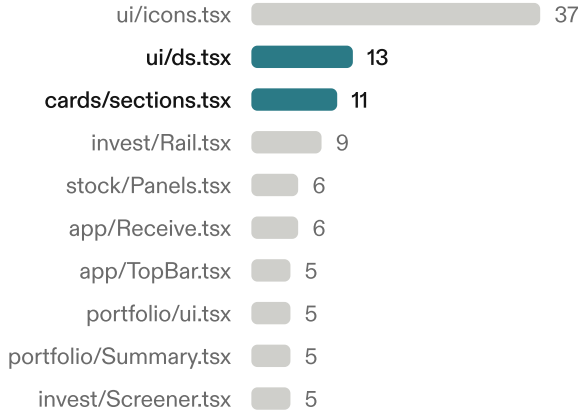
Bottom line. The code is in better shape than a strict rulebook assumes. Types are tight, most comments explain *why*, and server data already flows through React Query without prop-drilling. The problems are concentrated: two very large components (`BorrowSheet` , `AuthModal`), a handful of files holding many components, some duplicated hook code, and shared building blocks sitting inside feature folders.

Largest files (lines)

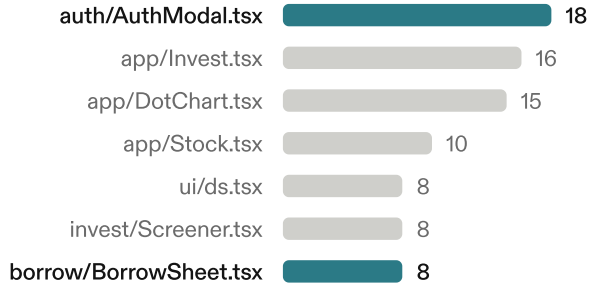


Highlighted: the four files this report focuses on. 87 source files, 12,355 lines in total.

Components per file



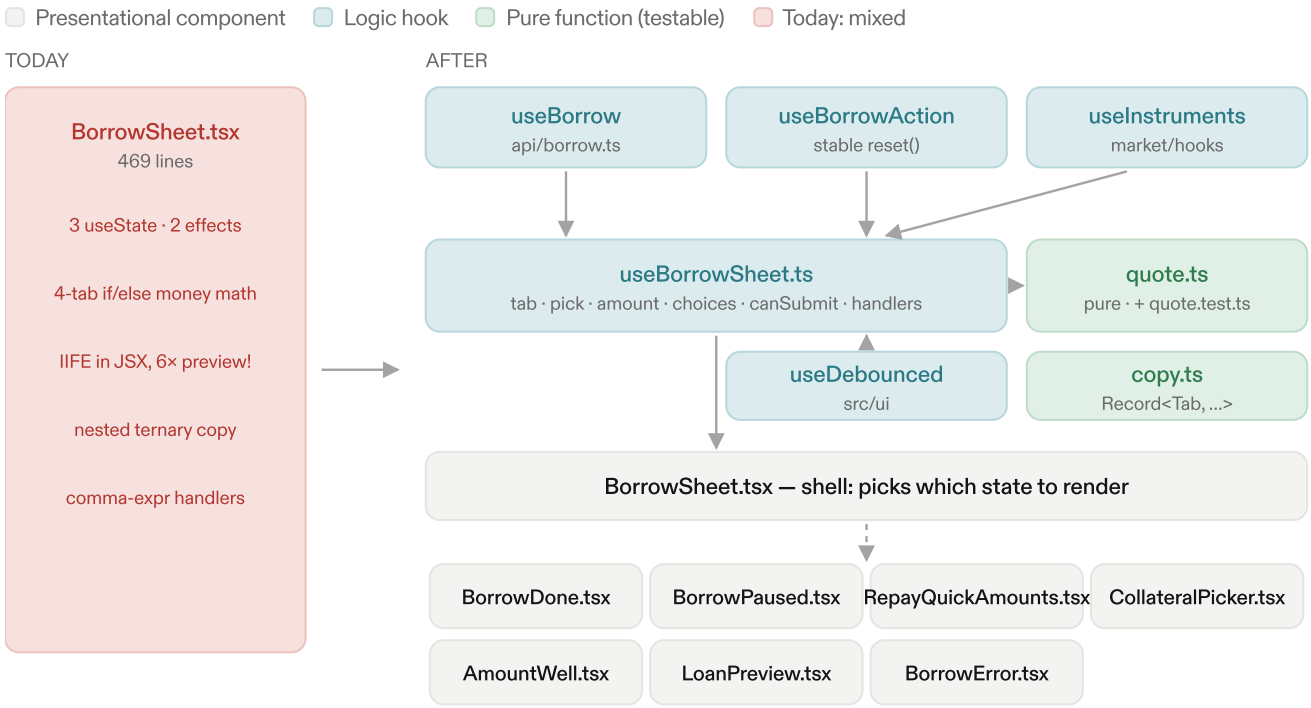
State and effect hooks per file



1 · LOGIC BLOAT & HOOK EXTRACTION

BorrowSheet.tsx

The Kamino borrow, repay, pledge and withdraw panel. The most important money math in the app sits in the render body and has no tests.



What's cluttering it

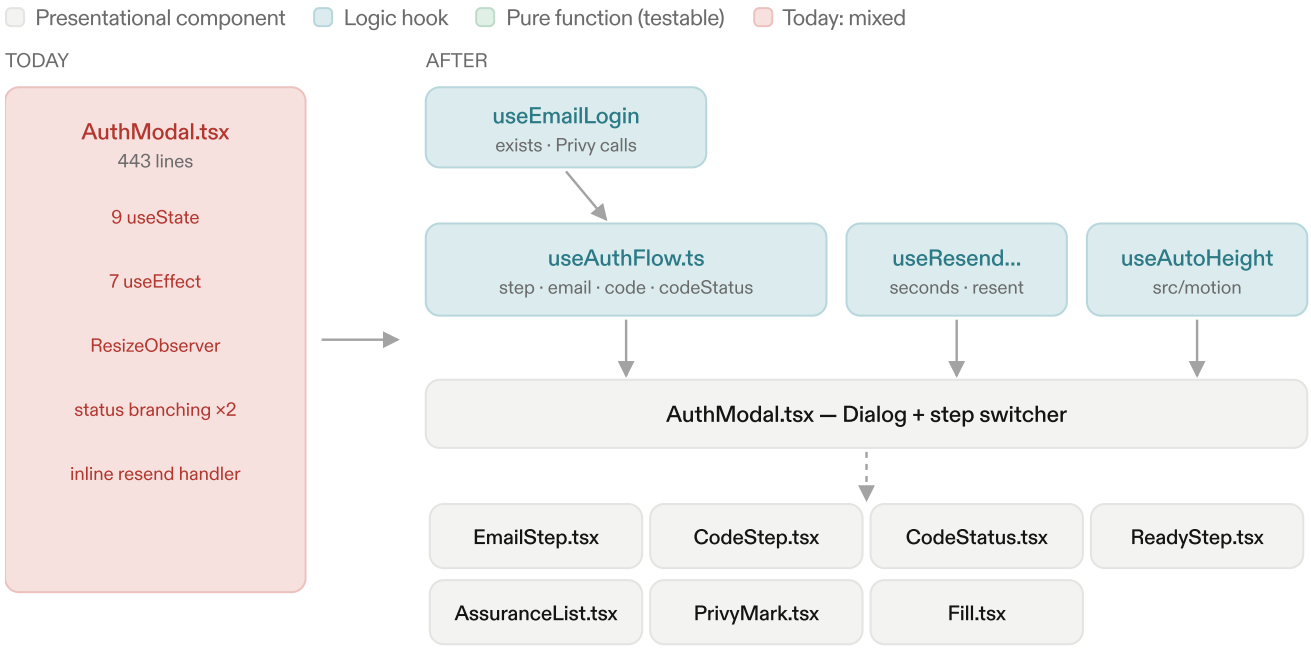
WHERE	PROBLEM	FIX
lines 108–182	An `if/else` over four tabs builds `intent`, `max`, `preview` and `blocked` from mutable `let`s	`quote.ts`, one pure function per tab, with tests
lines 383–429	An immediately-invoked function inside the JSX forces six `preview!` non-null assertions	`LoanPreview` receives a `preview` that is already known to exist
lines 247, 252, 312	Comma-expression handlers: `() => (setTab(…), setAmount(…))`	Named handlers from the hook
189–198, 445–457	Nested ternaries for the title, the button label and the empty-state copy	`copy.ts` lookup tables keyed by `Tab`
lines 68–76	A dependency is left out on purpose because `reset()` is a new function every render	Make `reset` stable in `useBorrowAction`
line 434	An `instanceof Error` check that can never fail (React Query v5 already types it)	Delete

Imports reaching into other features: `Mark` ← invest/shared, `RailLine` ← stock/Panels. Both move to `src/ui/` (see §2).

1 · LOGIC BLOAT & HOOK EXTRACTION

AuthModal.tsx

The email sign-in dialog. The Privy calls are already in `useEmailLogin.ts`, which is good; the modal then adds its own flow state, two timers and a layout animation on top.



Extraction map

NEW FILE	TAKES OVER
`auth/useAuthFlow.ts`	`step`, `email`, `invalid`, `code`, `submitEmail`, `resend`, the six-digit auto-submit, the 250 ms reset after close, and one derived `codeStatus`
`auth/useResendCountdown.ts`	`seconds`, `resent`, `restart()`, and the two timer effects
`motion/useAutoHeight.ts`	The `ResizeObserver` and the height/clip timer. It has nothing to do with sign-in, and other sheets can reuse it.
`motion/Fill.tsx`	The progress-bar fill, which is animation rather than sign-in

The status text is written out **twice** today, once for screen readers (297–305) and once on screen (309–362). A single `codeStatus` value from the hook feeds both.

Smaller duplicates found along the way

- **Close on outside click and Escape** is written three times: `Select`, `HeaderChrome`, `AccountMenu`. It becomes one `ui/useDismiss.ts`.
- **`useFlag`** is private to `Receive.tsx` and becomes `ui/useFlag.ts`.
- **Casts that should be type guards:** `as Tab` (Panels.tsx:61), `as Group` (Activity.tsx:201), `as Health` (borrow.ts:77). DOM casts like `e.target as Node` are fine.
- **Unused code:** `useMarketStatus` in `api/hooks.ts` is exported but never imported.

2 · MULTI-COMPONENT FILE SPLIT

src/ui/ds.tsx → src/ui/ds/

13 components used by 19 files. One file each; a temporary `index.ts` re-exports them so the split commit changes no importing file. Private constants travel with their component (`Tone` and `CHIP` go with StatusChip; `PITCH` / `BAR` with SegmentMeter; `pageList` becomes a pure `pagination.ts`).

RailHead.tsx 11 importers	StatusChip.tsx 9 importers	MoveChip.tsx 7 importers	InfoTip.tsx 6 importers	RailCard.tsx 5 importers
GlideRows.tsx 5 importers	RangeSwitch.tsx 4 importers	RailFigure.tsx 4 importers	SegmentMeter.tsx 2 importers	Pagination.tsx 2 importers
Chevron.tsx 2 importers	BookmarkMark.tsx 2 importers	LiveChip.tsx 1 importer	RailLine.tsx moved in from stock/	index.ts temporary barrel

cards/sections.tsx → cards/sections/

- CardHero · Rates · HowItWorks · TwoModes
- EarnWhileYouSpend · Plans · Platinum
- NonCustodial · CardClosing
- **GetCard** and **StoreMark** (shared by the hero and the closing)

Moved into src/ui/ (used by more than one feature)

- `RailLine` ← stock/Panels
- `Mark` ← invest/shared, renamed `InstrumentMark`
- `Skeleton` ← portfolio/ui
- `LoanMeter` stays in money/: it is a loan concept

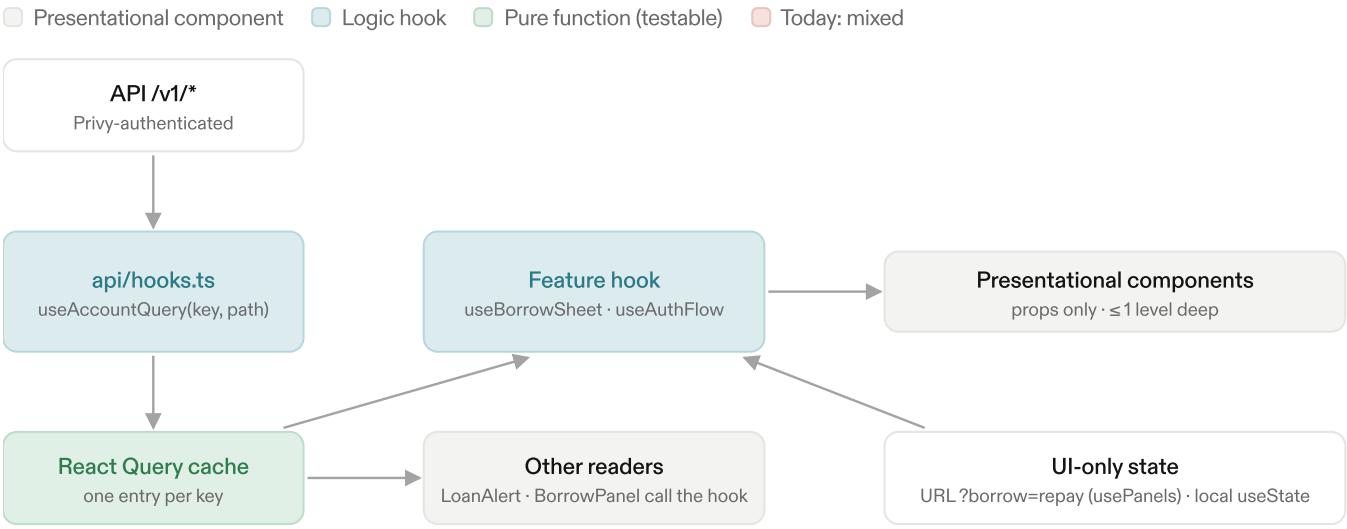
Second wave

FILE	BECOMES
invest/Rail.tsx (9)	invest/rail/: Saved, Fold, RailList, ListLoading, ListEmpty, Watchlist, Movers, MostBought, Sectors
stock/Panels.tsx (6)	Stats, FigureCard, Fact, Position, About
app/Receive.tsx (6)	app/receive/: QrBlock, CopyAddress, ShareAddress, DepositAddress, Receive, Swap
portfolio/ui.tsx (5)	Panel, Delta, Meter, Figure. `relDay` and `shortDate` go to `format.ts`
invest/shared.tsx	`points` and `compact` → chart-points.ts; `Row` → invest/types.ts
4–5 each	TopBar, Screener, BuySheet, Summary, Feed, HexAllocation, Top, Activity, Dialog, Login
ui/icons.tsx (37)	Suggested exception An icon set is conventionally one file. 37 tiny files would add churn without making anything clearer.

3 · SERVER STATE WITHOUT ZUSTAND

How data reaches the UI

React Query shares one cached request per key, so any component can call the hook it needs without props passing through the tree. Feature hooks combine queries and local state; presentational components only receive props.



Already right: keep

- `useBorrow()` is called in 6 files and fetched once
- No server data is copied into `useState`
- Which side panel is open lives in the URL (`usePanels`), so Back closes it

Rules for the refactor

- Derive data, don't store it: `useMemo` or a query's `select`
- Hooks own the data; components get props at most one level deep
- One `useAccountQuery` replaces the `useSignedIn()` + `enabled` setup repeated in five hooks
- One `queryKeys` object instead of key strings typed in several places

Is Zustand needed anywhere?

STATE	WHAT HAPPENS NOW	FIX	ZUSTAND?
Sign-in modal	`AuthProvider` is set up separately in AppShell, Login and Cards, so each page has its own copy, and `GlobalPanels` can't open it	One provider in `App.tsx` , or a URL parameter `?signin=1`	No
Watchlist	Each `useWatchlist` keeps its own copy of what's in the browser's storage. Two lists on one screen would drift apart; today Stock and Invest are separate pages, so it doesn't happen.	Only if needed: `useSyncExternalStore` , or Zustand with `persist`	Only here
Panels, forms	URL state and local `useState`	Keep as they are	No

4 · SAFE EXECUTION

Step by step

The working tree is clean; the last work was committed as `03ea847`. Work happens on `refactor/web-deslop` with one commit per step, and each step must pass the type-check, tests and build plus a visual check of the screens it touched before the next one starts.

0. **Branch + baseline**
   typecheck · tests · screenshots
1. **Shared hooks**
   useDismiss · useFlag · useDebounced · useAccountQuery
2. **quote.ts + tests** — manual QA · money
   pin current money math
3. **BorrowSheet** — manual QA · money
   useBorrowSheet + 8 components
4. **AuthModal** — manual QA · sign-in
   useAuthFlow + 7 components
5. **Split ds.tsx**
   src/ui/ds/ behind barrel
6. **Split the rest**
   sections · Rail · Panels · Receive…
7. **One AuthProvider**
   hoist to App.tsx
8. **Cleanup**
   drop barrels · narration comments

Decisions for you before starting

- ☐ Keep `ui/icons.tsx` as one file?
- ☐ Should small helpers used by only one component (like `PageChevron`) get their own files too?
- ☐ Temporary `index.ts` re-exports, or update every importing file in each split commit?
- ☐ Sign-in modal: one shared `AuthProvider`, or a `?signin=1` URL parameter?

Line numbers refer to commit 03ea847. Counts come from reading and searching the source in apps/web/src.